

Week 7: Data Integration

F20BD/F21BD — Big Data Management

Heriot-Watt University Dubai — 2025–26

Lecturer: Dr Drishty Sobnath

FINAL EXAM REVISION — BEGINNER-FRIENDLY

Key Concept

What this entire week is about (one paragraph):

Most real companies don't have one neat database. They have *many* databases scattered across different departments—Sales has one, HR has another, Marketing yet another. These isolated databases are called **data silos**, and they cause huge problems: duplicated data, inconsistent records, and wasted money. The big question this week answers is: “How do we glue all that scattered data together so we can actually use it?” We study several approaches. At one extreme, you can **physically copy everything** into one giant **data warehouse** (using ETL). At the other extreme, you can leave all data where it is and use a clever **virtual mediator** to query across sources on the fly (VDI). Finally, we see how the **Semantic Web** solves this problem elegantly via **OBDA** (Ontology-Based Data Access), where an OWL ontology acts as a “smart dictionary” on top of traditional SQL databases, letting you query them with SPARQL without moving a single byte of data.

Contents

1	Data Silos — The Problem (Exam Favorite!)	3
1.1	What Is a Data Silo?	3
1.2	Why Do Silos Form? (5 Causes)	3
1.3	Problems Caused by Silos (EXAM IMPORTANT!)	4
2	Data Integration — The Solution (Overview)	4
2.1	Six Main Data Integration Approaches	4
3	Data Warehouse Architecture (EXAM IMPORTANT!)	5
3.1	What Is a Data Warehouse?	5
3.2	Architecture Diagram	5
3.3	Key Components Explained	6
3.4	Two Warehouse Design Philosophies	6
3.5	Commercial Warehouse Products	6
4	Star Schema vs Snowflake Schema (EXAM IMPORTANT!)	6
4.1	Fact Tables and Dimension Tables	6
4.2	Star Schema	7
4.3	Snowflake Schema	7
4.4	Star vs Snowflake — Exam Comparison Table	7
5	ETL — Extract, Transform, Load (EXAM IMPORTANT!)	7
5.1	The Three Steps — Detailed	8
5.2	ETL vs ELT	8
6	Virtual Data Integration — VDI (EXAM IMPORTANT!)	8

6.1	VDI Architecture Diagram	9
6.2	How VDI Works — Step by Step	9
6.3	Four Key Characteristics of VDI	9
6.4	Advantages and Disadvantages	9
7	Mediator Architecture — Wrappers & Mappings (EXAM IMPORTANT!)	10
7.1	What Is a Wrapper?	10
7.2	What Are Mappings?	10
8	Mapping Approaches: GAV, LAV, GLAV (MOST IMPORTANT!)	10
8.1	GAV — Global-As-View (1990s)	10
8.2	LAV — Local-As-View (mid-1990s)	11
8.3	GLAV — Global-and-Local-As-View	11
8.4	GAV vs LAV vs GLAV — Exam Comparison Table	11
9	Semantic Data Integration & OBDA (EXAM IMPORTANT!)	12
9.1	The Problem That OBDA Solves	12
9.2	OBDA — Ontology-Based Data Access	12
9.3	How OBDA Works — Step by Step	12
9.4	OBDA Architecture Diagram	13
9.5	Why OBDA Matters — 5 Key Exam Points	13
10	Database-to-Ontology Mapping (EXAM IMPORTANT!)	13
10.1	1. Direct Mapping (Automatic, No Configuration)	13
10.2	2. R2RML (RDB-to-RDF Mapping Language)	14
10.3	Direct Mapping vs R2RML — Comparison	14
11	Ontop Framework & Protégé — Lab Practical	14
11.1	Lab Workflow — Step by Step	15
11.2	University Ontology Class Hierarchy (from <code>university.ttl</code>)	15
11.3	Object and Data Properties	16
11.4	Example OBDA Mapping — Line by Line	16
11.5	The <code>university.properties</code> Connection File	16
12	Master Comparison: Data Warehouse vs VDI (EXAM IMPORTANT!)	17
12.1	All Integration Approaches at a Glance	17
	EXAM PREPARATION	18
12.2	Key Terms Glossary	18
12.3	Common Exam Question Types	18
12.4	Practice Questions with Model Answers	19
12.5	Quick Revision Checklist	21

Data Silos — The Problem (Exam Favorite!)

What Is a Data Silo?

Picture a large supermarket chain called **FreshMart**. Over the years, different departments independently built their own computer systems:

- **Sales** tracks products sold, revenue, and trends — stored in a MySQL database.
- **HR** tracks employees, salaries, and attendance — stored in Oracle.
- **Marketing** tracks ad campaigns and click rates — stored in Google Sheets.
- **Customer Service** logs complaints and refunds — stored in a CRM tool (Salesforce).
- **Production** manages inventory and suppliers — stored in SAP.

Each system works perfectly *on its own*, but **none of them can talk to each other**. The Sales team can't see HR data. Marketing can't access Sales figures. That is what a data silo looks like in practice.

Key Concept

Official definition: “Repositories of information that are useful to the business unit that creates them but **inaccessible** to other teams that might benefit from the data they hold.” — Oracle, 2025

Real-world analogy: Imagine five roommates in a flat. Each one has their own mini-fridge locked in their bedroom. Everyone has food, but nobody knows what anyone else has. If one roommate runs out of milk, they go buy more—even though another roommate has a full carton about to expire. That is a data silo in everyday terms: useful stuff locked away where others can't see or use it.

Why Do Silos Form? (5 Causes)

Cause	Plain English Explanation
Organic growth	Each department builds its own system over years; nobody plans ahead for sharing.
Different technologies	HR picks Oracle, Sales picks MySQL, Marketing uses Excel. They literally can't connect.
Company culture	Departments compete rather than cooperate; “our data is our competitive advantage.”
Mergers & acquisitions	Two companies merge, but each keeps its old separate computer systems running.
Lack of governance	No company-wide rules exist about where to store data or how to share it.

Problems Caused by Silos (EXAM IMPORTANT!)

Problem	What It Means	FreshMart Example
No overview	Teams can't see each other's data	Marketing can't see which products sell best to target their ads
Duplication	Same data stored in multiple places	Customer "John Smith" appears in Sales, HR, Marketing, and CRM databases
Inconsistency	Copies get out of sync	Sales says John lives in London; HR says Dubai; Marketing says Paris
Wasted resources	Money and storage wasted on duplicates	Four copies of the entire customer list = 4× the storage cost
Poor decisions	Conflicting data causes wrong choices	Products shipped to the wrong address because of mismatched records

• EXAM TIP:

If the exam says “*Identify problems caused by data silos,*” remember the mnemonic **D–I–N–W–P**: **D**uplication, **I**nconsistency, **N**o overview, **W**asted resources, **P**oor decisions. Pick any two or three and explain with an example — that is worth full marks.

Data Integration — The Solution (Overview)

Key Concept

Data Integration = The process of combining data from several separate sources into **one unified view** so it can be analysed, reported on, or mined for insights. It is the direct solution to the data silo problem.

Analogy: Going back to the five-roommates-with-mini-fridges scenario—data integration is like either: (a) moving all the food from five mini-fridges into **one big shared kitchen fridge** (= data warehouse), or (b) putting a **whiteboard on the kitchen wall** listing exactly what's in every mini-fridge without moving any food (= virtual integration).

Six Main Data Integration Approaches

#	Approach	One-Sentence Summary
1	Manual/Middleware	Humans or middleware software manually link systems.
2	Data Federation	Virtual unified view; data stays in place.
3	Data Warehousing	Physically copy ALL data into one central database using ETL.
4	Application-Based	Individual apps pull from multiple sources at runtime.
5	Data Virtualisation	A smart “mediator” creates a virtual unified view using wrappers.
6	ETL / ELT	Extract → Transform → Load: the process that fills a warehouse.

• EXAM TIP:

The single most important distinction in this entire week: **Data Warehouse = data is physically COPIED and MOVED** to a central place. **Virtual Integration = data STAYS at its original source**. Every comparison question on the exam boils down to this one difference. Write it on your hand before you walk in.

Data Warehouse Architecture (EXAM IMPORTANT!)

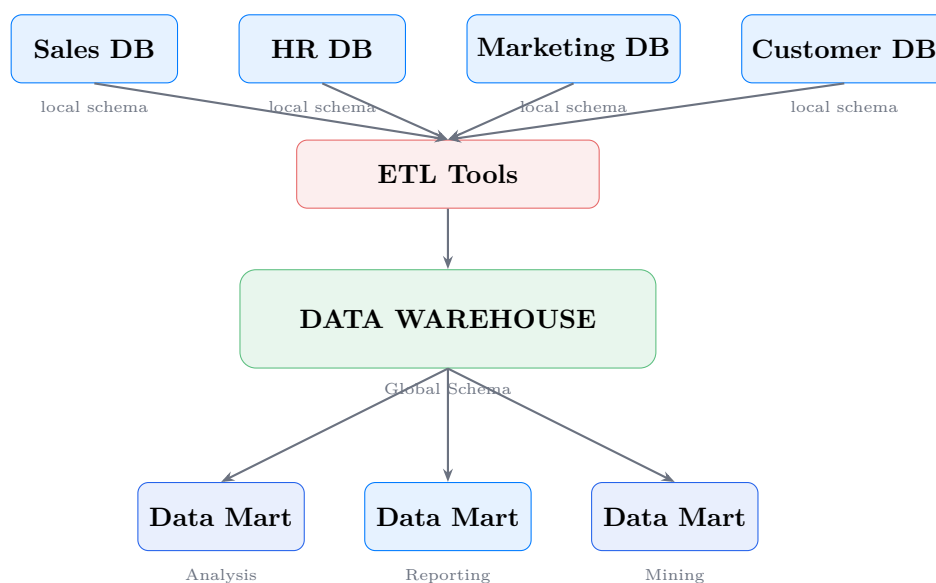
What Is a Data Warehouse?

Think of a data warehouse as a **giant central library**. Instead of keeping books scattered across five rooms (silos), you photocopy every book and put all copies in one central library. Now *anyone* can walk in and search for anything.

Key Concept

A data warehouse makes **periodical physical copies** of data from every source department and stores them in **one integrated central database** with a unified structure (global schema).

Architecture Diagram



Key Components Explained

Component	What It Is
Local Schema	Each source database has its own internal structure (table names, columns, data types). These are all different from each other.
ETL Tools	Software that connects to every source, extracts data, transforms it into a common format, and loads it into the warehouse.
Global Schema	The unified table structure inside the warehouse that combines all sources into one coherent design.
Data Mart	A smaller, focused subset of the warehouse designed for a specific team or topic (e.g., a “Sales Mart” containing only sales-related data).

Two Warehouse Design Philosophies

Approach	Description	When to Use
Normalised (Bill Inmon)	Tables follow 3NF with minimal redundancy; ensures data integrity and accuracy.	Enterprise-wide warehouses where accuracy is paramount.
Dimensional (Ralph Kimball)	Uses Fact and Dimension tables organised in Star or Snowflake shapes; optimised for fast querying.	Departmental data marts and BI dashboards where query speed matters most.

Commercial Warehouse Products

Amazon **Redshift** • Google **BigQuery** • Microsoft **Synapse** • IBM **Db2** • Oracle Autonomous DW • **Snowflake**

Star Schema vs Snowflake Schema (EXAM IMPORTANT!)

These are the two most common ways to organise tables *inside* a dimensional data warehouse.

Fact Tables and Dimension Tables

Fact table = stores the *measurable numbers* (the “what happened?” data):

sale_id	product_id	date_id	store_id	amount_sold
1	42	20250301	5	\$120.00
2	17	20250301	3	\$45.50

The numeric column (**amount_sold**) is the **measure**. The ID columns are **foreign keys** linking to dimension tables.

Dimension tables = answer the context questions (**WHO, WHAT, WHEN, WHERE**):

product_id	name	category	brand
42	Wireless Mouse	Electronics	Logitech
17	Notebook A5	Stationery	Moleskine

Star Schema

(Sales); [databox, above=of fact] (time) Time; [databox, left=of fact] (prod) Product; [databox, right=of fact] (store) Store; [databox, below=of fact] (cust) Customer; [arr] (time) – (fact); [arr]

(prod) – (fact); [arr] (store) – (fact); [arr] (cust) – (fact);

Each dimension is a **single flat table** (denormalised). The fact table sits in the centre connected directly to each dimension → forming a star shape. **Pros:** simple, fast, few JOINS. **Cons:** redundant data in dimensions.

Snowflake Schema

(Sales); [databox, above=of fact, minimum width=1.8cm] (day) Day; [databox, above=of day, minimum width=1.8cm] (month) Month; [databox, above=of month, minimum width=1.8cm] (year) Year; [databox, left=1.5cm of fact, minimum width=1.8cm] (prod) Product; [databox, left=of prod, minimum width=1.8cm] (cat) Category; [databox, right=1.5cm of fact, minimum width=1.8cm] (city) City; [databox, right=of city, minimum width=1.8cm] (country) Country; [databox, below=of fact, minimum width=1.8cm] (cust) Customer; [arr] (day) – (fact); [arr] (month) – (day); [arr] (year) – (month); [arr] (prod) – (fact); [arr] (cat) – (prod); [arr] (city) –

(fact); [arr] (country) – (city); [arr] (cust) – (fact);

Dimensions are **split into normalised sub-tables**, creating branches (snowflake shape). **Pros:** less redundancy, finer drill-down (Year→Quarter→Month). **Cons:** more JOINS, slower, more complex.

Star vs Snowflake — Exam Comparison Table

• EXAM TIP:

Memorise this table. It appears in almost every exam:

Feature	Star Schema	Snowflake Schema
Dimension structure	Single flat tables	Split into normalised sub-tables
Complexity	Simple	More complex
Data redundancy	Higher (denormalised)	Lower (normalised)
Number of JOINS	Fewer → faster queries	More → slower queries
Storage efficiency	Less efficient	More efficient
Best used for	Data marts and simple reporting	Detailed hierarchical drill-down

ETL — Extract, Transform, Load (EXAM IMPORTANT!)

ETL is the **three-step process** used to fill a data warehouse with data from multiple sources.

The Three Steps — Detailed

Read from sources; [procbox, right=1cm of e, minimum width=3.8cm] (t) **2. Transform**
 Clean, merge, convert; [mainbox, right=1cm of t, minimum width=3.8cm] (l) **3. Load**

Write to warehouse; [arr] (e) – (t); [arr] (t) – (l);

Step	What Happens	Beginner Analogy
1. Extract	Read raw data from each source database. You need a specific software connector for each type (MySQL connector, Oracle connector, CSV reader, etc.).	Going to 5 different public libraries and photocopying all their books.
2. Transform	Convert extracted data into the warehouse's common format. Includes: <i>cleaning</i> (fix typos, remove duplicates), <i>merging</i> (combine records), <i>applying rules</i> (convert GBP to USD), <i>calculating</i> (compute totals and averages).	Translating all the photocopied books into one common language and fixing any torn or illegible pages.
3. Load	Write the cleaned, unified data into the warehouse according to its target schema.	Placing all the translated, repaired books neatly onto the shelves of one big central library.

ETL vs ELT

	ETL	ELT
Full form	Extract → Transform → Load	Extract → Load → Transform
Where transformed?	On a <i>separate</i> ETL server	<i>Inside</i> the target database itself
Best for	Traditional on-premise warehouses	Modern cloud warehouses (BigQuery, Snowflake) with powerful built-in processing

Tools: Open source: Apache NiFi, Talend. Commercial: Informatica, IBM DataStage, Microsoft SSIS.

Virtual Data Integration — VDI (EXAM IMPORTANT!)

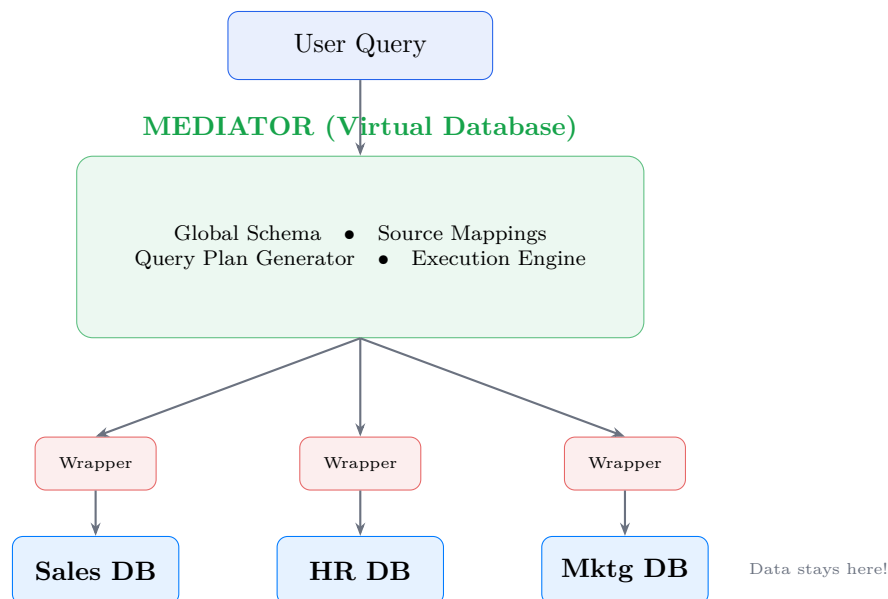
Key Concept

In VDI, data **stays where it is** (no copying, no moving). A **mediator** provides a virtual unified view by breaking your query into sub-queries, sending each to the right source via **wrappers**, and combining the results.

Analogy: Instead of photocopying all books into one library (warehouse), you simply *phone*

each library, ask your question, and combine the answers yourself. The books never move.

VDI Architecture Diagram



How VDI Works — Step by Step

1. You write **one query** against the global schema (the unified virtual view).
2. The **mediator** receives your query.
3. The mediator uses **mappings** to figure out which data lives in which source database.
4. The **query planner** breaks your query into sub-queries, one per relevant source.
5. Each sub-query is sent through a **wrapper** (connector/translator) to the appropriate source.
6. Each source database runs its sub-query locally and returns results to the mediator.
7. The mediator **combines** all partial results and returns the complete answer to you.

Four Key Characteristics of VDI

1. **Sources remain independent** — each database is updated on its own schedule.
2. **Sources can join multiple systems** — the same HR database can participate in VDI System A and VDI System B simultaneously.
3. **Sources can be added or removed** without rebuilding the whole system.
4. **Mediated schema captures relationships** between all sources and the global view.

Advantages and Disadvantages

	Advantages	Disadvantages
Data	Always up-to-date (you query live data, never stale)	If a source goes offline, its data becomes completely unavailable
Cost	No massive storage costs (zero duplication)	Complex query planning logic inside the mediator
Flexibility	Easy to add or remove sources	Network latency degrades performance
Independence	Sources remain fully autonomous	Slower than querying a local warehouse

Mediator Architecture — Wrappers & Mappings (EXAM IMPORTANT!)

What Is a Wrapper?

A **wrapper** is a software connector that sits between a data source and the mediator. Think of it as a **bilingual translator**: the mediator speaks one “language” and the database speaks another — the wrapper translates both ways.

Key Strengths / Features

Key facts about wrappers:

- Wraps the source so the mediator can communicate with it
- Can present a **different schema** than the actual source (transforms the view)
- The same source can have **different wrappers** for different integration systems
- Data returned can be **transformed** at the wrapper level before reaching the mediator
- Exports data from the source **as requested** by the mediator

Real-world analogy: Your mediator is an English-only manager. Your database is a French-only employee. The wrapper is their bilingual colleague who translates questions from English to French and answers from French back to English. If you have a German-only database too, you need a *different* wrapper (a German translator).

What Are Mappings?

Mappings are logical formulas (usually SQL queries or views) that tell the mediator **exactly where each piece of data lives** and how it relates to the global schema.

- They describe **what data** is available at each source
- They relate **local schemas** → **global (mediated) schema**
- They are the **basis for generating query plans** — the mediator reads mappings to decide where to route each piece of a query

Analogy: Mappings are like a phone directory. The mediator looks up “customer name” and finds: “*stored in the Sales database, table ‘clients’, column ‘full_name’*.” That tells the mediator exactly where to go.

Mapping Approaches: GAV, LAV, GLAV (MOST IMPORTANT!)

• EXAM TIP:

This is one of the **most commonly examined** topics. You **MUST** know all three approaches and be able to compare them in a table. This is almost **guaranteed** to appear on the exam.

GAV — Global-As-View (1990s)

Key Concept

In GAV, the **global schema is defined as views over the local schemas**. You look at what each source database has, and you write a “recipe” (view) that combines them into the global view.

Restaurant analogy: You look at what ingredients your 3 suppliers have, then write the menu based on what they can provide. If a *new* supplier joins, you have to **rewrite the entire menu**.

- Global view *depends on* local sources
- ✓ **Easy to answer queries** — just follow the view definitions to find data
- × **Hard to add new sources** — the global view must be rewritten every time a source changes

LAV — Local-As-View (mid-1990s)

Key Concept

In LAV, the **local schemas are defined as views over the global schema**. You first design the ideal unified schema (the “dream database”). Then each source says “I can provide THIS part of your dream.”

Restaurant analogy: You first write your dream menu. Then each supplier says “I can provide items 3, 7, and 12 from your menu.” Adding a new supplier is easy — they just list what menu items they can support.

- Local sources describe themselves *in terms of* the global schema
- ✓ **Easy to add new sources** — just describe what the new source provides
- × **Harder to answer queries** — the mediator must figure out which combination of sources can answer each query

GLAV — Global-and-Local-As-View

Key Concept

In GLAV, mappings are expressed as **correspondences between views on BOTH the global schema AND the local schemas**. Both GAV and LAV can be derived as special cases of GLAV.

GAV vs LAV vs GLAV — Exam Comparison Table

• EXAM TIP:

Memorise this table — it is worth 4–6 marks on its own:

Feature	GAV	LAV	GLAV
Full name	Global-As-View	Local-As-View	Global-and-Local-As-View
Era	Named in 1990s	Mid-1990s	Later
Global is...	Views over local	Independent	Correspondence
Local is...	Independent	Views over global	Correspondence
Adding sources	Hard (rewrite global)	Easy	Flexible
Query answering	Easy	Harder	Balanced
Relationship to GLAV	Special case	Special case	Most general

Semantic Data Integration & OBDA (EXAM IMPORTANT!)

The Problem That OBDA Solves

Companies have **massive volumes of existing data** sitting in traditional relational (SQL) databases. Converting all of it into RDF triple stores would be **incredibly expensive** and time-consuming — we're talking years of work and millions in costs among large organisations.

But we *want* to use Semantic Web technologies (SPARQL queries, OWL reasoning, linked data). So the question becomes: **how do we get Semantic Web benefits without moving any data?**

The answer: put an ontology on TOP of the existing database.

OBDA — Ontology-Based Data Access

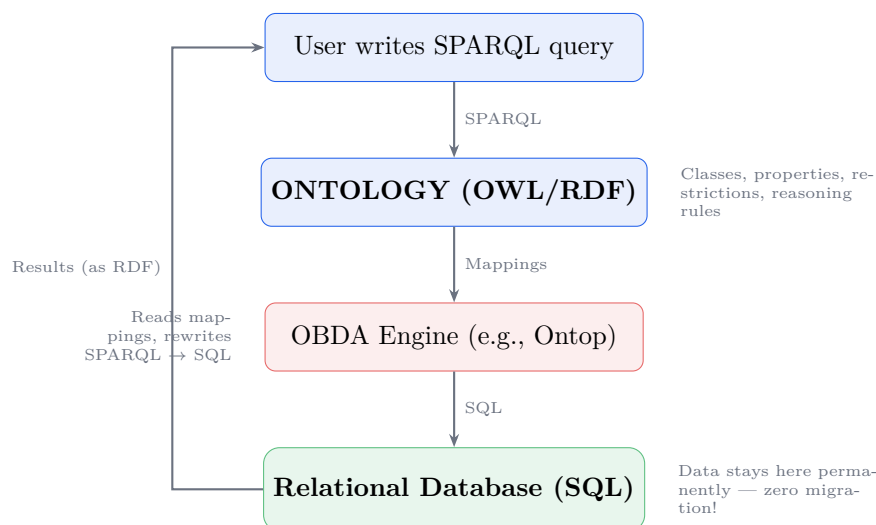
Key Concept

OBDA = Use an OWL ontology as a **high-level global schema** over existing relational databases. Users query with **SPARQL** using friendly ontology vocabulary. The system **automatically rewrites** the SPARQL into SQL and runs it on the database. The user **never needs to know** the database structure. Data **never moves**.

How OBDA Works — Step by Step

1. **Define an ontology** (in OWL) describing the domain — e.g., “there are Students, Courses, Teachers; Students attend Courses; Teachers teach Courses.”
2. **Create mappings** linking ontology concepts to actual database tables — e.g., “the class :Student maps to SQL table uni1.student.”
3. A user **writes a SPARQL query** using ontology terms — e.g., “Find all Students and their first names.”
4. The **OBDA engine** (e.g., Ontop) reads the mappings and **rewrites the SPARQL into SQL**: `SELECT * FROM uni1.student.`
5. The SQL runs on the **existing relational database**; results come back.
6. The engine formats results **as if they were RDF triples** and returns them to the user.

OBDA Architecture Diagram



Why OBDA Matters — 5 Key Exam Points

Key Strengths / Features

1. **No data migration** — data stays in the relational database, saving huge costs and effort
2. **Users don't need SQL** — they query with SPARQL using friendly ontology vocabulary
3. **Ontology hides complexity** — users don't see ugly database table/column names
4. **Reasoning/inference** — OWL ontology enables logical deductions (e.g., “every Professor is also a Researcher”)
5. **Reuses existing SQL infrastructure** — leverages the speed, scale, and features of mature RDBMS systems

Database-to-Ontology Mapping (EXAM IMPORTANT!)

There are **two W3C standards** for connecting relational databases to RDF:

1. Direct Mapping (Automatic, No Configuration)

A **straightforward, rule-based, fully automatic** approach. You give it a database and it generates RDF with zero configuration:

Database Element	Becomes in RDF
Table	OWL Class (e.g., <code>student</code> table → <code>:student</code> class)
Column	Datatype Property (e.g., <code>name</code> column → <code>:name</code> property)
Row	Individual instance (e.g., row with ID 42 → <code>:student/42</code>)
Foreign Key	Object Property (relationship between two individuals)

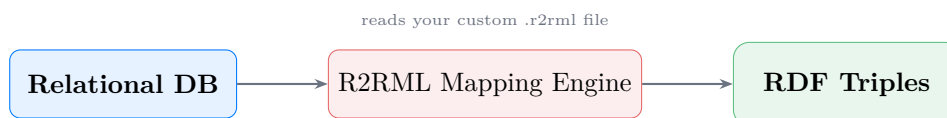
✓ **Zero configuration** — fully automatic, just point it at a database

- × **No customisation** — you're stuck with the database's naming and structure
- × Output mirrors the raw database schema, which may not be meaningful

2. R2RML (RDB-to-RDF Mapping Language)

Key Concept

R2RML is a W3C standard for **customisable** mapping of relational databases to RDF. You write mapping rules (**triples maps**) that define exactly how each row in a table becomes RDF triples. The R2RML file passes through a **Mapping Engine** to produce RDF output.



Direct Mapping vs R2RML — Comparison

Feature	Direct Mapping	R2RML
Customisation	None — fully automatic	Full control over output
W3C Standard?	Yes	Yes
Effort required	Zero	Must write mapping definitions
Output quality	Basic (mirrors DB schema)	Production-quality, meaningful RDF
Best use case	Quick prototyping and demos	Real-world semantic data integration

Ontop Framework & Protégé — Lab Practical

Lab / Practical

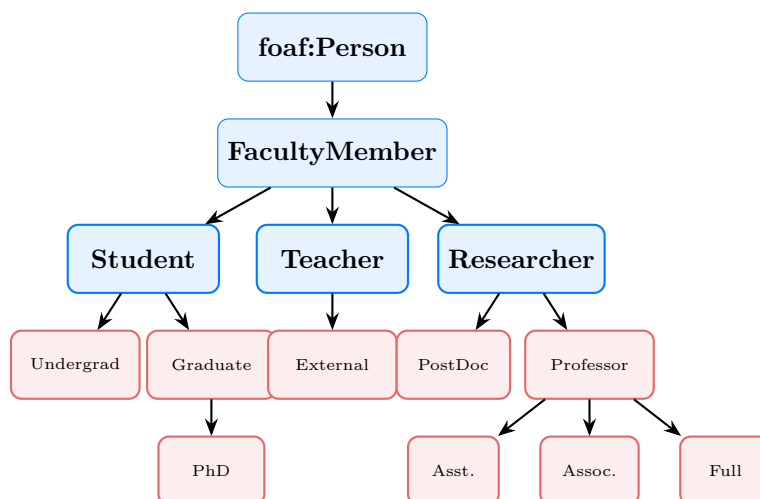
The lab uses three tools together to perform OBDA hands-on:

1. **Protégé** — free ontology editor from Stanford University
2. **Ontop plugin** — OBDA engine that plugs into Protégé
3. **H2 Database** — lightweight SQL database containing university data

Lab Workflow — Step by Step

Step	Action	What It Means
1	Define Ontology	In Protégé, create OWL classes (Student, Course, Professor...), properties (teaches, attends...), and relationships between them.
2	Connect to DB	Point Ontop at the H2 database using the <code>university.properties</code> JDBC configuration file (driver, URL, username, password).
3	Create Mappings	Write OBDA mappings in <code>university.obda</code> that link ontology concepts to SQL tables.
4	Query with SPARQL	Run SPARQL queries in Protégé; Ontop automatically rewrites them into SQL and returns results as RDF.

University Ontology Class Hierarchy (from `university.ttl`)



Critical relationships to remember for the exam:

- `Professor` is a subclass of *both* `Researcher` AND `Teacher` (multiple inheritance)
- `PhDStudent` is a subclass of *both* `GraduateStudent` AND `Researcher`
- `GraduateStudent` has an OWL restriction: must be supervised by *some* `Professor` (`isSupervisedBy some Professor`)
- `isTaughtBy` is the **inverse** of `teaches`
- `givesLecture` and `givesLab` are **sub-properties** of `teaches`
- `Course`, `EducationalInstitution`, and `Person` are pairwise **disjoint** (nothing can be both a `Course` and a `Person`)

Object and Data Properties

Property	Domain	Range	Type
teaches	Teacher	Course	Object
isTaughtBy	Course	Teacher	Object (inverse of teaches)
attends	Person	Course	Object
isGivenAt	Course	EducationalInstitution	Object
isSupervisedBy	—	—	Object
givesLecture	—	—	Object (sub of teaches)
givesLab	—	—	Object (sub of teaches)
firstName	Person	Literal	Data
lastName	Person	Literal	Data
title	—	xsd:string	Data

Example OBDA Mapping — Line by Line

```
[MappingDeclaration] @collection [[
  mappingId    uni1-student
  target       :uni1/student/{s_id} a :Student ;
               foaf:firstName {first_name}^^xsd:string ;
               foaf:lastName  {last_name}^^xsd:string .
  source       SELECT * FROM "uni1"."student"
]]
```

What each line does:

Line	Plain English Explanation
mappingId	Gives this mapping a human-readable name: “uni1-student”
source	The SQL query that fires: “grab every row from the uni1.student table”
:uni1/student/{s_id}	For each row, create an RDF individual whose IRI uses the student’s primary key (e.g., :uni1/student/42)
a :Student	Declare that individual as <code>rdf:type :Student</code>
foaf:firstName	Attach the <code>first_name</code> column value as a <code>foaf:firstName</code> data property, typed as <code>xsd:string</code>
foaf:lastName	Same for <code>last_name</code>

The university.properties Connection File

```
jdbc.driver=org.h2.Driver
jdbc.url=jdbc:h2:tcp://localhost/./university-session1
jdbc.user=sa
jdbc.password=
```

This tells Ontop **how to connect** to the H2 database: which JDBC driver to use, the database URL, the username (`sa` = system admin), and an empty password.

Master Comparison: Data Warehouse vs VDI (EXAM IMPORTANT!)

• EXAM TIP:

This table is the **single most useful comparison** to memorise. It covers all the key differences in one place:

Feature	Data Warehouse	VDI (Virtual)
Data location	Copied to central DB	Stays at original source
Freshness	Can be stale (periodic re-fresh)	Always current (live queries)
Storage cost	High (storing duplicate copies)	Low (no duplication)
Query speed	Fast (data is local)	Slower (network queries to sources)
Source availability	Works even if sources go offline	Fails if a source goes offline
Adding new sources	Need to build new ETL pipeline	Just add a new wrapper
Core mechanism	ETL process	Mediator + Wrappers + Mappings

All Integration Approaches at a Glance

Approach	Data Moved?	Real-Time?	Complexity
Data Warehouse	Yes (copied)	No (batch ETL)	High setup
ETL/ELT	Yes (part of DW)	No (batch)	Medium
VDI	No	Yes	High (mediator)
OBDA	No	Yes	Medium
Data Federation	No	Yes	Medium

EXAM PREPARATION

Key Terms Glossary

Term	Definition
Data Silo	Isolated data repository accessible only to the team that created it
Data Integration	Combining multiple data sources into one unified view
Data Warehouse	Central database that stores physical copies of data from all sources
ETL	Extract → Transform → Load: the process that fills a warehouse
VDI	Virtual Data Integration: unified view without moving data
Mediator	Central component in VDI that routes queries to sources
Wrapper	Connector/translator between a source and the mediator
Mapping	Logical formula relating local schemas to the global schema
GAV	Global-As-View: global schema defined as views over local schemas
LAV	Local-As-View: local schemas defined as views over global schema
GLAV	Most general mapping approach; GAV and LAV are special cases
OBDA	Ontology-Based Data Access: OWL ontology over SQL databases
Direct Mapping	Automatic DB-to-RDF conversion (no config needed)
R2RML	W3C standard for customisable DB-to-RDF mapping
Star Schema	Fact table + flat dimension tables (denormalised)
Snowflake Schema	Fact table + normalised dimension sub-tables
Fact Table	Stores measurable data (sales amounts, quantities)
Dimension Table	Stores context (WHO, WHAT, WHEN, WHERE)
Ontop	OBDA framework that rewrites SPARQL → SQL
Protégé	Stanford's free OWL ontology editor

Common Exam Question Types

Based on the lecture and lab material, expect these question styles:

1. **Define and explain** — “What is a data silo?” / “What is OBDA?”
2. **Compare two approaches** — “Compare DW vs VDI” / “Compare Star vs Snowflake” / “Compare GAV vs LAV”
3. **Describe a process** — “Explain the three steps of ETL” / “How does Ontop process a SPARQL query?”
4. **Interpret a mapping** — Given an OBDA mapping snippet, explain what it does
5. **Draw/describe architecture** — “Draw the VDI mediator architecture”

Practice Questions with Model Answers

Exam-Style Question

Q1. Define “data silo” and explain two problems it causes. [4 marks]

Model Answer: A data silo is an isolated repository of data that is accessible only to the department that created it and inaccessible to other teams that might benefit from it (Oracle, 2025). **Problem 1 — Data duplication:** the same information (e.g., customer names and addresses) is stored in multiple separate systems, wasting storage resources and maintenance effort. **Problem 2 — Data inconsistency:** when the same data exists in multiple places, copies inevitably go out of sync (e.g., Sales says a customer lives in London while HR says Dubai), leading to conflicting information and poor business decisions.

Exam-Style Question

Q2. Compare Data Warehousing and Virtual Data Integration. Give two advantages of each. [6 marks]

Model Answer: *Data Warehousing* physically extracts data from all sources using ETL, transforms it into a common format, and loads it into one central database. **Advantage 1:** Fast query performance because all data is stored locally in one place. **Advantage 2:** The warehouse continues to function even if the original source databases go offline. *Virtual Data Integration* keeps data at its original sources and uses a mediator to provide a virtual unified view by routing queries live to each source. **Advantage 1:** Data is always up-to-date because queries hit live sources (no stale copies). **Advantage 2:** No additional storage costs because data is never duplicated.

Exam-Style Question

Q3. Explain the three steps of ETL. [3 marks]

Model Answer: **Extract:** Read raw data from multiple source databases using specific software connectors (e.g., MySQL connector, Oracle connector). **Transform:** Convert the extracted data to match the target warehouse’s schema — this includes cleaning errors, merging duplicate records, applying business rules, converting formats, and calculating derived values. **Load:** Write the cleaned and transformed data into the data warehouse according to its target schema.

Exam-Style Question

Q4. Compare GAV and LAV mapping approaches. [4 marks]

Model Answer: In **GAV** (Global-As-View), the global schema is defined as views over the local source schemas — the global unified view is built by describing how to combine what each source provides. This makes *query answering easy* (just follow the view definitions) but *adding new sources is hard* (the global view must be rewritten). In **LAV** (Local-As-View), each local source schema is described as a view over the global schema — each source says which part of the ideal global schema it can provide. This makes *adding new sources easy* (just describe what they offer) but *query answering is harder* (the system must determine which combination of sources can satisfy the query).

Exam-Style Question

Q5. What is OBDA? Explain how Ontop processes a SPARQL query. [6 marks]

Model Answer: OBDA (Ontology-Based Data Access) is a semantic data integration approach where an OWL ontology serves as a high-level global schema over existing relational databases. Users query using SPARQL without needing to understand the underlying database structure.

Ontop processes a SPARQL query as follows: (1) The user submits a SPARQL query using the ontology’s vocabulary (e.g., “find all Students”). (2) Ontop reads the OBDA mappings, which link ontology concepts (e.g., `:Student`) to database tables (e.g., `uni1.student`). (3) Ontop automatically rewrites the SPARQL query into an equivalent SQL query. (4) The SQL query is executed against the relational database. (5) Results are returned to the user formatted as RDF triples. Crucially, no data migration is needed — all data remains in the original relational database.

Exam-Style Question

Q6. Compare Star Schema and Snowflake Schema. [4 marks]

Model Answer: A **Star Schema** has a central fact table linked directly to flat, de-normalised dimension tables, forming a star shape. It is simple to understand and query (fewer JOIN operations needed), making it fast and well-suited for data marts and basic reporting. However, dimension tables can contain redundant data.

A **Snowflake Schema** extends the star by normalising dimension tables into multiple sub-tables, creating a branching structure resembling a snowflake. This reduces data redundancy and enables finer drill-down analysis (e.g., Year → Quarter → Month), but queries are more complex and slower due to the increased number of JOIN operations.

Exam-Style Question

Q7. Interpret the following OBDA mapping: [4 marks]

```
target  :uni1/student/{s_id} a :Student ;
        foaf:firstName {first_name}^^xsd:string .
source  SELECT * FROM "uni1"."student"
```

Model Answer: The `source` line selects every row from the SQL table `uni1.student`. For each row, the `target` creates an RDF individual with the IRI `:uni1/student/<s_id>` where `s_id` is the student’s primary key from the database (e.g., `:uni1/student/42`). It declares this individual as `rdf:type :Student`. It then assigns the `foaf:firstName` datatype property using the value from the `first_name` column, typed as `xsd:string`. This mapping effectively “virtualises” SQL rows as RDF triples without physically moving any data.

Exam-Style Question

Q8. What is the role of a wrapper in Virtual Data Integration? [3 marks]

Model Answer: A wrapper is a software connector (translator) that sits between a data source and the mediator in a VDI system. It translates the mediator’s requests into a format the data source understands, and translates the source’s responses back into the format expected by the mediator. A wrapper can present a *different schema* than the source’s actual structure, effectively transforming how the data appears to the mediator. The same data source can have different wrappers for participation in different virtual integration systems.

Quick Revision Checklist

Before the exam, make sure you can confidently answer “yes” to each:

✓?	Can You...
☐	Define “data silo” and list 4 problems it causes?
☐	Name at least 4 data integration approaches?
☐	Draw the Data Warehouse architecture (sources → ETL → DW → marts)?
☐	Compare Star vs Snowflake schema in a table?
☐	Explain all 3 steps of ETL with examples?
☐	Explain the difference between ETL and ELT?
☐	Draw the VDI mediator architecture (user → mediator → wrappers → sources)?
☐	Explain what wrappers and mappings do?
☐	Compare GAV, LAV, and GLAV in a table?
☐	Define OBDA and explain how Ontop works step-by-step?
☐	Compare Direct Mapping vs R2RML?
☐	Read and interpret an OBDA mapping from a .obda file?
☐	Compare Data Warehouse vs VDI with at least 5 features?

End of Week 7 — Data Integration

Good luck on your exam!